# Open-Source LLMs — Student Cheat Sheet (Day 2 · v2 · July 2026)

**Bootcamp:** Full-Stack & AI Bootcamp 2026 · Week 16 · Day 2 **Audience:** Full-Stack devs · GenAI students · Data Analytics students **Pin this near your screen.**

---

## The July 2026 model map (WILL be stale by September — the questions won't)

| Family | Flagship (Jul 2026) | License | Note |
|---|---|---|---|
| GLM (Zhipu) | GLM-5.2 · 753B/40B MoE · 1M ctx | MIT | #1 open-weights model |
| Kimi (Moonshot) | K3 · 2.8T · 16-of-896 experts | open | largest open model ever |
| DeepSeek | V4.1 Pro & Flash · 1M ctx | MIT | Flash weights open, Pro API-only |
| Qwen (Alibaba) | 3.5/3.6 open · 3.7 API-only | Apache 2.0 | family SPLIT open/closed |
| Llama (Meta) | Llama 4 Scout/Maverick (Apr 2025) | Llama lic. | LEGACY — Meta went closed |
| Gemma (Google) | Gemma 4 · 5 sizes E2B→31B | Apache 2.0 | best laptop family |
| MiniMax | M3 · SWE-bench Pro 59.0 | open | agentic + multimodal |

**8-week lesson:** between May and July 2026: GLM-5.2 took #1, Kimi K3 shipped at 2.8T, Meta exited open frontier, Qwen split. Any table like this has a shelf life measured in weeks.

## What is a "weight"? (the 30-second version)

A weight = **one learned number** (like `0.0273`). It sets how strongly one signal influences another inside the model. "7B model" = 7 billion of them.

Nobody writes them — **training finds them**: start random → guess the next word → nudge the numbers that would have made the right word likelier → repeat trillions of times. Everything the model "knows" is just where the numbers landed.

**Consequences:** file size = count × bytes per number · facts are smeared across numbers (no lookup table — that's why "if you don't tell it, it doesn't know") · quantization = rounding each number slightly (7B of them voting together, quality barely moves).

## The 2026 open-source frontier (at a glance)

| Family | Flagship (May 2026) | Architecture | Context | License | Best for |
|---|---|---|---|---|---|
| **DeepSeek** | V4-Pro (1.6T total / 49B active) · V4-Flash (284B / 13B) | MoE | 1M | **MIT** | Reasoning, code, the "challenger" leader |

| Family | Flagship (May 2026) | Architecture | Context | License | Best for |
|---|---|---|---|---|---|
| **Llama** (Meta) | Llama 4 — Maverick (400B / 17B), Scout (109B / 17B) | MoE | up to 10M (real ~500K) | Llama (custom, MAU cap) | Mature ecosystem, biggest community |
| **Qwen** (Alibaba) | Qwen 3.5 — sizes 0.6B → 235B+ | MoE + dense | 256K | **Apache 2.0** | Multilingual, broadest size ladder |
| **Mistral** | Mistral Large 3 (675B / 41B), Mistral Small 3, Codestral | MoE + dense | 128K | **Apache 2.0** (most) | EU data residency, code |
| **Gemma** (Google) | Gemma 4 — sizes 2B → 27B | dense | 128K | **Apache 2.0** | Smaller, runs on a single GPU |
| **Kimi** | Kimi K2.6 | MoE | long | open weights | Currently top of LMArena open-only |

**Live leaderboards to bookmark:** LMArena · Open LLM Leaderboard · llm-stats.com · Artificial Analysis.

---

## Picking a model — six questions

1. **Task** — chat, code, classify, multilingual, reasoning?
2. **License** — Apache 2.0 (do anything), MIT (do anything), Llama (commercial OK with MAU cap), restrictive (read carefully)?
3. **Hardware** — your laptop, a single GPU, a cluster? Determines size.
4. **Latency** — interactive UI or batch? Smaller is faster.
5. **Quality bar** — does it have to beat GPT-5 on your task? Most often it doesn't have to.
6. **Hosting story** — local, self-hosted on your cloud, or a hosted open provider (Together / Groq / Fireworks)?

   **Default heuristic:** start with the medium-tier model in any frontier family. Only reach for the flagship (or scale down to the SLM) when a specific task makes you.

---

## MoE in one paragraph

A **Mixture of Experts** model has a huge total parameter count but only routes each token through a small fraction ("active params"). DeepSeek V4-Pro is 1.6T total with 49B active — costs and speeds are determined by the **active** count, not the total. Almost every flagship open model in 2026 is MoE because it's the cheapest way to scale quality.

---

## Reasoning models in open-source

The reasoning-model trick (model thinks internally before answering) is now in the open-source world too:

- **DeepSeek-R1 lineage** (R1, R1-Zero, R1-Distill) — the first widely-available open reasoning model.
- **Qwen QwQ / Qwen 3.5 thinking** — Alibaba's reasoning track.
- **Llama 4 reasoning** — Meta's variant.

Same Day 1 rule applies: **don't add "let's think step by step" prompts to a reasoning model.** They reason internally; you'll just pay for duplicate tokens.

---

## Tools for running a model — pick by job

| Tool | Best for | What it gives you |
|---|---|---|
| **Ollama** | Solo developer, prototyping, "I want a local API in 30 seconds" | One command (`ollama pull qwen3:0.6b`), OpenAI-compatible API on localhost:11434 |
| **LM Studio** | Beginners, GUI users, machines without a discrete GPU | Click-to-run UI, often beats Ollama on integrated GPUs |
| **vLLM** | Production serving, multi-user concurrency | ~6× throughput vs Ollama at concurrency, PagedAttention, NVIDIA GPUs only |
| **llama.cpp** | The engine under Ollama and LM Studio | Direct GGUF inference, what you reach for to optimize further |
| **Hosted open providers** | "I want to use Llama 4 from a browser, no install" | Together AI, Groq, Fireworks AI, Hugging Face Inference — OpenAI-compatible APIs |

---

## Quantization in one paragraph

Quantization compresses model weights (e.g. from float16 down to 4-bit integers) so a model that would need 80 GB of VRAM at full precision fits in 24 GB at Q4. The trade is a small accuracy loss. Two formats matter:

- **GGUF** — the format Ollama / LM Studio / llama.cpp use. Common quants: `Q4_K_M`, `Q5_K_M`, `Q8_0`. **`Q4_K_M` is the most common default.**
- **AWQ / GPTQ** — used in vLLM and Hugging Face for GPU inference.

Rule of thumb: **a 7B model at Q4 needs roughly 4–5 GB of RAM/VRAM.**

---

## Hosted open providers — the "no install" path

When you can't run locally (no GPU, no time), use a hosted provider running open-source models:

| Provider | Strengths |
|---|---|
| **Together AI** | Wide model catalog, OpenAI-compatible API |
| **Groq** | Insanely fast inference (custom LPU chips), free tier |
| **Fireworks AI** | Production-grade, fine-tuning support |
| **Hugging Face Inference** | Largest model selection, integrates with HF Hub |
| **Cerebras** | Very fast inference, growing model catalog |

All four expose the same OpenAI-shaped chat endpoint. Same prompt, change the base URL and the model name, you've moved providers.

---

## Mini-lab — pick your bot (Day 2 style)

You'll do **two** hands-on segments today:

### Hands-on #1: Cross-model bake-off (hosted)

In a hosted provider playground (Together AI / Groq / Hugging Face), run the **same prompt** on three different open models. Compare:

- **Output quality** — same task, different prose / different correctness
- **Latency** — how long did it take?
- **Cost** — input + output token cost

Pick three models that interest you. Suggested triple: `meta-llama/Llama-4-Maverick`, `deepseek-ai/DeepSeek-V4-Flash`, `Qwen/Qwen3.5-235B`.

### Hands-on #2: Ollama + Python (local)

```
# 1. Install Ollama (https://ollama.com), then:
ollama pull qwen3:0.6b      # ~520 MB, runs on any laptop
ollama serve                # starts the API on localhost:11434
```

Then a 30-line Python script:

```
import json, requests

def ask(prompt, model="qwen3:0.6b", system=None):
    payload = {
        "model": model,
        "messages": (
            ([{"role": "system", "content": system}] if system else [])
            + [{"role": "user", "content": prompt}]
        ),
        "stream": False,
        "options": {"temperature": 0},
    }
    r = requests.post("http://localhost:11434/v1/chat/completions",
                      json=payload, timeout=120)
    return r.json()["choices"][0]["message"]["content"]

# Reuse the Day 1 6-ingredient prompt frame!
SYSTEM = "You output ONLY JSON. Schema: {\"sentiment\":\"pos|neg|neu\",\"confidence\":<float>}
print(ask("Loved the coffee, hated the wait.", system=SYSTEM))
```

That is a real LLM. Running on your machine. With no API key.

---

## Agent frameworks — the 2026 lineup

| Framework | What it's for | Status |
|---|---|---|
| **LangGraph** | LangChain's stateful agent successor. Graphs of nodes + edges. | Most actively developed |
| **CrewAI** | Multi-agent "crews" — researcher, coder, reviewer, etc. | Easiest multi-agent start (LangGraph overtook it in stars) |
| **OpenAI Agents SDK** | OpenAI's official SDK for tool-using agents | New, well-documented |
| **Pydantic AI** | Type-safe agent framework, structured outputs first-class | Growing fast |
| **LangChain (classic)** | Still huge ecosystem, still relevant for chains | Active but recommend LangGraph for new agent work |
| **MCP (Model Context Protocol)** | Not a framework — the standard wiring between models and tools | Adopted by Anthropic, OpenAI, IDEs. Bootcamp covers later this week. |

**Dropped from older curricula:** TaskWeaver (Microsoft de-prioritized), JARVIS (research project, not used in production).

**Decision shortcut:**

- One agent doing one job? — **OpenAI Agents SDK** or **Pydantic AI**.
- One agent following a graph of steps? — **LangGraph**.
- Multiple agents collaborating? — **CrewAI**.
- Need to plug into Claude Desktop / Cursor / IDEs? — **MCP**.

---

## Where to go next

- **Fine-tuning** — bootcamp's separate fine-tuning module. LoRA / QLoRA are the techniques to know.
- **MCP day** — Week 16's MCP day deep-dives the agent-tool wiring layer.
- **Eval at scale** — same tooling as Day 1: Langfuse, Promptfoo, Braintrust.
- **Production inference** — vLLM, TGI, SGLang. Read their docs once per project.

**Canonical references:**

- **Hugging Face Hub** — the central catalog for every model on this sheet.
- **LMArena** — human-judged head-to-head leaderboard.
- **Open LLM Leaderboard** — automated benchmark leaderboard.
- **Ollama model library** — `ollama.com/library`.
- **llm-stats.com** — live cost / latency / quality comparisons across providers.

---

*Cheat sheet · Open-Source LLMs · Day 2 · May 2026*

---

## Homework — the ladder continues (self-paced, ~3.5h)

The `homework/` folder in your handout pack has the full written workshop + 8 runnable scripts. Class ended at "swap the model"; homework continues:

- `04_chat_app.py` — chat with MEMORY (build the messages list yourself — Day 1's context window, hand-built)
- `05_thinking_mode.py` — actually RUN a reasoning model
- `05b_langchain_intro.py` + `06_agent.py` — your first LangChain + LangGraph agent

Start with: `python homework/examples/verify_setup.py`